

De evolucionar robots a entrenar modelos

Clase 5: recapitulacion y primer paso con PyTorch

Laboratorio de Inteligencia Artificial

1. Recapitulacion 10 min

La historia completa:
representacion, evaluacion y
busqueda.

2. Conexion 7 min

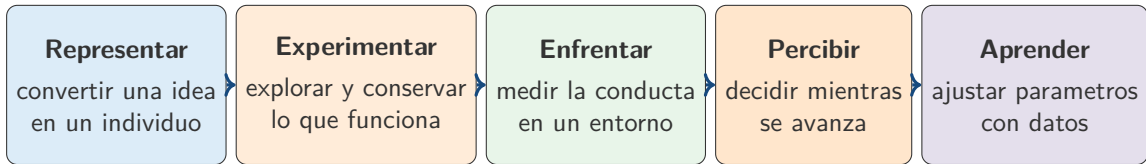
Que cambia cuando la regla
deja de ser escrita por
nosotros.

3. PyTorch 10 min

Entorno, instalacion,
verificacion y un tensor.

Pregunta de salida

¿Que parte del robot evolutivo queremos que aprenda un modelo?



La idea que permanece

Definimos una representacion, medimos su calidad y usamos esa medida para mejorar soluciones.

Problema

Un robot parte de $(0,0)$ y quiere llegar a $(7,7)$.

U D L R

El ADN es una secuencia de movimientos.

Regla de supervivencia

$$fitness = 100 - distancia$$

La poblacion conserva las mutaciones que mejoran el puntaje.

Leccion

El algoritmo no sabe que significa “bien”. Nosotros lo expresamos en la funcion de fitness.

Exploracion

Mutacion alta, mas diversidad y mas regiones probadas. Riesgo: mucho ruido y poca estabilidad.

Explotacion

Mutacion baja y elitismo alrededor de buenas soluciones. Riesgo: convergencia prematura.

buscar = descubrir + conservar lo que funciona

La ruta

Sigue siendo un plan completo de 18 comandos.

D D D L D R ...

Es un control *open loop*: no corrige durante el recorrido.

Fitness compuesto

$$500 - 35d + 4p - 30c - 3r + 2000m$$

Distancia, pasos utiles, choques, repeticiones y llegada.

Leccion

Una solucion puede acercarse a la meta y seguir siendo mala si choca demasiado.

Percepcion local

Cuatro bits en orden U D L R: celda libre o bloqueada.

$$2^4 = 16 \text{ observaciones}$$

El ADN tiene 16 reglas reutilizables.

Bucle de control

observar



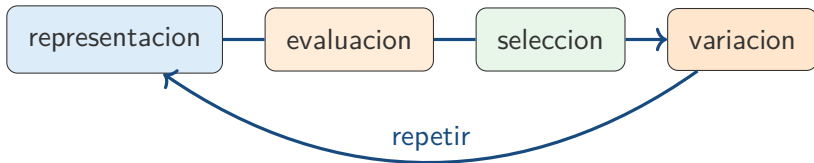
adn[observacion]



mover

Ganancia y limite

Gana adaptacion al obstaculo; pierde informacion global y puede entrar en ciclos.



Representar

¿Que es un individuo?

Medir

¿Que significa mejor?

Variar

¿Como aparecen alternativas?

La receta sirve para un ADN, una politica o los parametros de una red neuronal.

- 1 La representacion determina que soluciones son faciles o dificiles de encontrar.
- 2 El fitness introduce nuestros objetivos y tambien nuestros sesgos.
- 3 La diversidad protege la busqueda; el elitismo conserva avances.
- 4 Percibir cambia el control: de ejecutar una ruta a decidir en cada paso.
- 5 Un experimento reproducible necesita semillas, metricas y archivos guardados.

Puente

Hasta ahora escribimos el mecanismo de variacion. Con aprendizaje automatico, los parametros se ajustan a partir de datos.

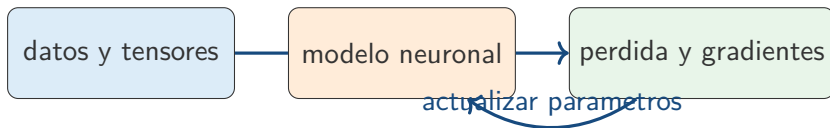
Robot evolutivo

- Individuo: ADN de comandos o reglas.
- Mejora: mutacion y cruce.
- Guia: fitness del entorno.
- Datos: simulaciones generadas.

Modelo neuronal

- Individuo: parametros de una red.
- Mejora: gradiente y retropropagacion.
- Guia: funcion de perdida.
- Datos: ejemplos de entrada y salida.

fitness y **perdida** cumplen papeles parecidos, pero no son el mismo algoritmo.



- Un tensor es un arreglo numerico con forma y tipo.
- PyTorch calcula gradientes automaticamente.
- La GPU es opcional para comenzar; CPU basta para el mini tutorial.

Linux o macOS

```
python3 -m venv .venv  
source .venv/bin/activate  
python -m pip install --upgrade pip
```

Windows PowerShell

```
py -m venv .venv  
.\.venv\Scripts\Activate.ps1  
python -m pip install --upgrade pip
```

El entorno virtual mantiene las dependencias de esta clase separadas de otros proyectos.

Instalacion inicial, CPU

```
python -m pip install torch torchvision torchaudio
```

¿Y si tengo NVIDIA?

Para CUDA, elegir el comando exacto en pytorch.org/get-started/locally: depende de la version compatible con el controlador. No mezclar al azar paquetes CPU y CUDA.

- En clase: empezamos por CPU para que todos tengan el mismo punto de partida.
- La instalacion puede tardar y descargar varios cientos de MB.

```
python -c "import torch; print(torch.__version__); \\
print(torch.rand(2, 3)); print(torch.cuda.is_available())"
```

Version

aparece una version

Tensor

aparece una matriz 2×3

CUDA

False es correcto en CPU

Si falla `import torch`, comprobar primero que el entorno virtual esta activo y que se uso el mismo comando `python` para instalar y ejecutar.

```
import torch

x = torch.tensor([[1.0, 2.0, 3.0],
                  [4.0, 5.0, 6.0]])

print(x.shape)
print(x + 1)
print(x[:, 0])
```

Lectura

Un tensor guarda numeros y conoce su forma. Aqui tenemos 2 filas y 3 columnas; podemos operar con todos sus valores de una vez y seleccionar una columna.

Representar → medir → mejorar → verificar

Ya podemos explicar

por que una politica llega,
choca o se cicla.

Ya podemos preparar

un entorno reproducible para
trabajar con tensores.

Siguiente paso

entrenar un modelo pequeno
y leer su perdida.

Salida rapida

En una frase: ¿que diferencia hay entre una regla escrita, una politica evolucionada y un modelo entrenado?